```
# a2a/message_protocol.py
from datetime import datetime
from typing import Dict, Any, Optional
from dataclasses import dataclass, field
import json

@dataclass
class A2AMessage:
    """Enhanced A2A message with agent card support"""
    message_id: str
    sender_agent_id: str
    recipient_agent_id: str
    message_type: str  # TASK_REQUEST, TASK_RESPONSE, AGENT_CARD, HEARTBEAT
    task_id: Optional[str] = None
    payload: Dict[str, Any] = field(default_factory=dict)
    sender_agent_card: Optional[Dict] = None  # Sender's agent card
    required_skills: List[str] = field(default_factory=list)
    timestamp: datetime = field(default_factory=datetime.utcnow)
    correlation_id: Optional[str] = None  # For request-response matching
    priority: int = 1
    expiration: Optional[datetime] = None

    def to_dict(self) -> Dict[str, Any]:
        """Convert message to dictionary for transmission"""
        return {
            "message_id": self.message_id,
            "sender_agent_id": self.sender_agent_id,
            "recipient_agent_id": self.recipient_agent_id,
            "message_type": self.message_type,
            "task_id": self.task_id,
            "payload": self.payload,
            "sender_agent_card": self.sender_agent_card,
            "required_skills": self.required_skills,
            "timestamp": self.timestamp.isoformat(),
            "correlation_id": self.correlation_id,
            "priority": self.priority,
            "expiration": self.expiration.isoformat() if self.expiration else None
        }

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> 'A2AMessage':
        """Create message from dictionary"""
        return cls(
            message_id=data["message_id"],
```

```
            sender_agent_id=data["sender_agent_id"],
            recipient_agent_id=data["recipient_agent_id"],
            message_type=data["message_type"],
            task_id=data.get("task_id"),
            payload=data.get("payload", {}),
            sender_agent_card=data.get("sender_agent_card"),
            required_skills=data.get("required_skills", []),
            timestamp=datetime.fromisoformat(data["timestamp"].replace('Z', '+00:00')),
            correlation_id=data.get("correlation_id"),
            priority=data.get("priority", 1),
            expiration=datetime.fromisoformat(data["expiration"].replace('Z', '+00:00')) if
data.get("expiration") else None
        )

class A2AProtocol:
    """Handles A2A message creation and routing"""

    def __init__(self):
        self.message_handlers = {}

    def create_task_message(self, task: Task, sender_agent_card: AgentCard) -> A2AMessage:
        """Create A2A message for task execution"""

        message_id = f"MSG-{task.task_id}-{str(uuid.uuid4())[:8]}"

        return A2AMessage(
            message_id=message_id,
            sender_agent_id=task.sender_agent_id,
            recipient_agent_id=task.destination_agent_id,
            message_type="TASK_REQUEST",
            task_id=task.task_id,
            payload={
                "task_name": task.task_name,
                "description": task.description,
                "input_data": task.input_data,
                "required_skills": task.required_skills,
                "priority": task.priority,
                "timeout_seconds": task.timeout_seconds
            },
            sender_agent_card=sender_agent_card.to_json(),
            required_skills=task.required_skills,
            correlation_id=task.task_id,
            priority=task.priority,
            expiration=datetime.utcnow() + timedelta(seconds=task.timeout_seconds)
```

```
        )

    def create_agent_card_message(self, agent_id: str, agent_card: AgentCard,
                                  recipient_agent_id: str = "broadcast") -> A2AMessage:
        """Create A2A message for agent card advertisement"""

        message_id = f"MSG-AGC-{str(uuid.uuid4())[:8]}"

        return A2AMessage(
            message_id=message_id,
            sender_agent_id=agent_id,
            recipient_agent_id=recipient_agent_id,
            message_type="AGENT_CARD",
            payload={
                "advertisement_type": "CAPABILITIES",
                "timestamp": datetime.utcnow().isoformat()
            },
            sender_agent_card=agent_card.to_json(),
            correlation_id=f"AGC-{agent_id}"
        )

    def create_task_response(self, original_message: A2AMessage, task: Task,
                             success: bool, output_data: Dict = None,
                             error_message: str = None) -> A2AMessage:
        """Create response message for task completion"""

        message_id = f"MSG-RES-{str(uuid.uuid4())[:8]}"

        return A2AMessage(
            message_id=message_id,
            sender_agent_id=task.destination_agent_id,
            recipient_agent_id=task.sender_agent_id,
            message_type="TASK_RESPONSE",
            task_id=task.task_id,
            payload={
                "success": success,
                "output_data": output_data or {},
                "error_message": error_message,
                "task_status": task.status.value,
                "processing_time": str(task.completed_at - task.started_at) if task.started_at and
task.completed_at else None
            },
            correlation_id=original_message.correlation_id
        )
```

```
# Global A2A protocol handler
a2a_protocol = A2AProtocol()
```